ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Lecture-11

Avinash K. Sharma
Asst. Professor, Department of CSE

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES (KCS301)

TOPIC: LINKED LIST: INTRODUCTION WITH SINGLY LINKED LIST

List

Objectives

- Describe a list
- How a list can be implemented by linked structure

List

- List is homogeneous collection of elements, with linear relationship between the elements, the list can be ordered or unordered.
- **Implementing list**

List can be implemented using

1. Linear array
2. Linked list

Array implementation of lists

- The linear array can be created at the compilation time by using type declaration statement of type

```
int a[100];
```

- Which create linear array with 100 elements
- Since each elements takes two bytes of memory, the compiler allocates 200 bytes for the array.
- The above array can be created at run time with the following declaration statement

```
int *a;
a=(int*)malloc(100*sizeof(int));
```

In the above declaration, the malloc() function allocates 200 bytes of memory and assign the address of the first byte to the pointer variable a.

Limitations of an array

- Insertion and deletion operations are expensive.
- Inserting at first position requires first pushing the entire array down by one position to make room.
- Deleting the first element requires shifting all the elements in the list one position up.
- So worst case of these operation is O(n).
- Size of the list must be known in advance.

Linked implementation of list

- To avoid the linear cost of insertion and deletion operations, we need to ensure that the elements of the list are not stored contiguously.
- Linked list basically consists of series of structures, which are not necessarily adjacent in memory
- Each structure contains an element of the list and a pointer to structure containing its successor.
- Linked implementation allow traverse and search operations to be carried out in linear time.
- Insertion and deletion operations can be implemented efficiently as it requires only rearrangement of pointers.

Linked list defined

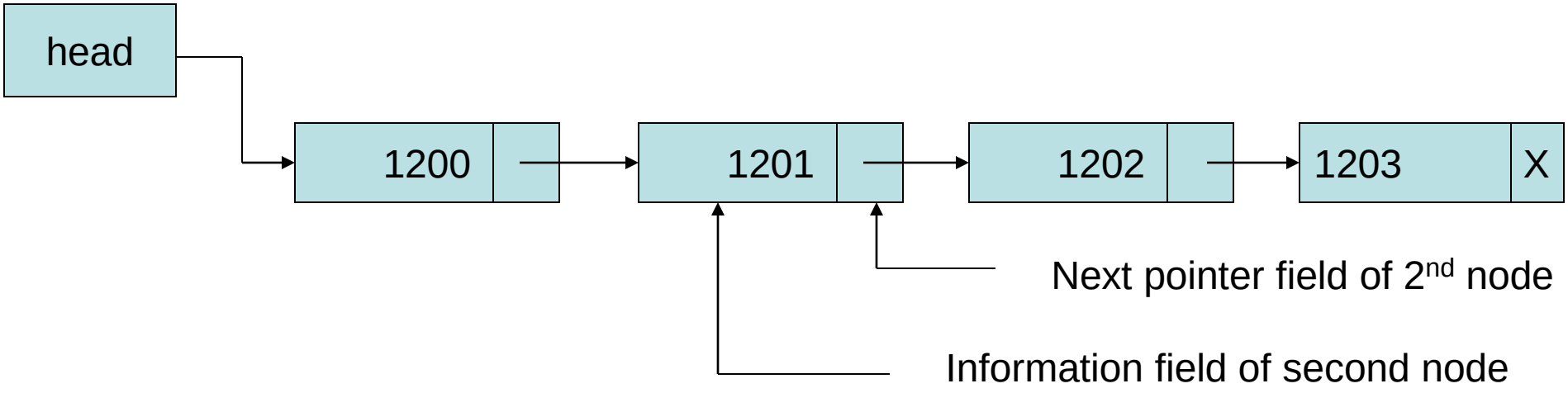
A linked list is a linear collection of data elements, called nodes, the linear order is given by pointers. Each node is divided into two or more parts. Linked list can be of following types:

- **Linear linked list or one way list**
- **Doubly linked list or two way list**
- **Circular linked list**

Linear linked list

In a linear linked list, also called singly linked list or one way linked list, each node is divided into two parts.

- First parts contain the information of the element
- Second part called the linked field or next pointer field, contains the address of the next node in the list.



•head is used to hold the address of first element of the list.

•Last element of the linked list have NULL value in the next pointer field to mark the end of the list

Representation of Linear linked list

Suppose we want to store the list of integer numbers, then the linear linked list can be represented in memory with the following declarations.

```
typedef struct nodetype
{
        int info;
        struct nodetype *next;
}node;
node *head;
```

The above declaration define a new data type, whose each element is of type nodetype and gives it a name node.

Operation on Linear linked lists

- **Creating an empty list**
- **Traversing a list**
- **Searching an element**
- **Inserting an element**
- **Deleting an element**

Creating an empty list

- In the previous declaration, the variable head is declared as pointer to node data type.
- Variable head is not yet given a value.
- This variable is used to point to the first element of the list
- Since the list will be empty in the beginning, the variable head is assigned a sentinel value to indicate the list is empty.

```
void createemptylist(node **head)
{
        *head=NULL;
}
```

Traversing a list

Linear list can be traversed in two ways

- In order traversal
- Reverse order traversal

In order traversal:

To traverse the linear linked list, we move along the pointer, and process each element till we reach the last element.

```
void traverseinorder(node *head)
{
        while(head!=NULL)
        {
                printf("%d\n",head->info);
                head=head->next;
        }
}
```

Traversing a list

Reverse order traversal:

To traverse the linear linked list in reverse order, we move along the pointer till we reach the last element. The last element is processed first, then the second last and so on and finally the first element of the list

To implement this we use either sack (LIFO) or recursion.

```
Void traversereverseorder(node *head)
        {
                if(head->next!=NULL)
                {
                        traversereverseorder(head->next);
                        printf("%d\n",head->info);
                }
        }
```

Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Lecture-14

Avinash K. Sharma
Asst. Professor, Department of CSE

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES (KCS301)

TOPIC: OPERATIONS ON LINKED LIST: INSERTION

Inserting an element

To insert an element in the list, the first task is to get a free node, assign the element to be inserted to the info field of the node, and then new node is placed at the appropriate position by adjusting the appropriate pointer. The insertion in the list can take place at the following positions:

- **At the beginning of the list**
- **At the end of the list**
- **After a given element**

Inserting an element

Insert at the beginning of the list:

First test whether the linked list is initially empty, if yes, then the element is inserted as the first and only one element by performing the following steps:

- Assign NULL value to the next pointer field of the new node
- Assign address of new node to head

If the list is not empty, then the element is inserted as the first element of the list by performing the following steps:

- Assign value of head variable to the next pointer field of the new node.
- Assign address of the new node to the head.

Inserting an element

Insert at the beginning of the list:

```
Void insertatbegining(node **head,int item)
  {
        node *ptr;
        ptr=(node*)malloc(sizeof(node));
        ptr->info=item;
        if(*head==NULL)
                ptr->next=NULL;
        else
                ptr->next=*head;
        *head=ptr;
  }
```

Inserting an element

Inserting at the end of the list:

First test whether the linked list is initially empty, if yes, then the element is inserted as the first and only one element by performing the following steps:

- Assign NULL value to the next pointer field of the new node
- Assign address of new node to head

If the list is not empty, then the list is traversed to reach the last element, and then element is inserted as the last element of the list by performing the following steps:

- Assign NULL value to the next pointer field of the new node
- Assign address of the new node to the next pointer field of the last node.

Inserting an element

```
Void insertatend(node **head, int item)
  {
        node *ptr, *loc;
         ptr=(node*)malloc(sizeof(node));
        ptr->info=item;
        ptr->next=NULL;
        if(*head==NULL)
                *head=ptr;
        else
        {
                loc=*head;
                while (loc->next!=NULL)
                        loc=loc->next;
                        loc->next=ptr;
        }
  }
```

Inserting an element

Inserting after given element:

To insert the new element after the given element, first we find the location, say loc, of the given element in the list, and then the element is inserted in the list by performing following steps:

- Assign the next pointer field of the node pointed by loc to the next pointer field of the new node.
- Assign address of the new node to the next pointer field of the node pointed by loc.

Inserting an element

```
Void insertafterelement(node *head, int item,int after)
{
    node *ptr, *loc;
    loc=search(head,after);
    if(loc==(node*)NULL)   /*element after not found*/
        return;
    ptr=(node*)malloc(sizeof(node));
    ptr->info=item;
    ptr->next=loc->next;
    loc->next=ptr;
}
```

Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Lecture-15

Avinash K. Sharma
Asst. Professor, Department of CSE

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES (KCS301)

TOPIC: OPERATIONS ON LINKED LIST: DELETION

Deleting an element

- To delete an element from the list, first the pointer are set properly and then the memory occupied by the node to be deleted is deallocated(free).
- The deletion in the list can take place at the following positions.

1. At the beginning of the list
2. At the end of the list
3. After a given element

Deleting from the beginning of the list

An element from the beginning of the lists can be deleted by performing following steps:

- Assign the value of head (address of the first element of the list) to a temporary variable (say ptr)
- Assign the value of the next pointer field of the first node to head.
- Deallocate the memory occupied by the node pointed to by ptr.

Deleting from the beginning of the list

```
Void deletefrombegining( node **head)
{
    node *ptr;
    if(*head==NULL)
        return;
    else
    {
        ptr=*head;
        *head=(*head)->next;
        free(ptr);
    }
}
```

Deleting from the end of the list

To delete from the end of the list, we first traverse to the second last element of the list. Then the last element can be deleted by performing following steps:

- Assign the next pointer field of the second last node to a temporary variable (say ptr).
- Assign value NULL to the next pointer field of the second last node of the list.
- Deallocate the memory occupied by the node pointed to by ptr.

Deleting from the end of the list

```
Void deletefromend( node **head)
{
    node *ptr,*loc;
    if (*head==NULL)
            return;
    else if ((*head)->next==(node*) NULL)
            {
            ptr=*head;
            *head=NULL;
            free(ptr);
            }
        else
            {
            loc=*head;
            ptr=(*head)->next;
            while(ptr->next!=NULL)
                {
                  loc=ptr;
                  ptr=ptr->next;
                }
             loc->next=NULL;
             free(ptr);
              }
}
```

Deleting after a given element

To delete an element after a given element, first we find the location say (loc) of the element after which the element can be deleted by performing the following steps:

- Assign next pointer field of the node pointed by the loc to temporary variable (say ptr).
- Assign the next pointer field of the node to be deleted to the node pointed to by loc
- Deallocate the memory occupied by the node pointed to by ptr.

Deleting after a given element

```
Void deleteafterelement( node*head, int after)
{
   node *ptr, *loc;
   loc=search(head,after);
   if(loc==(node*)NULL)    /*element ‘after’ not found*/
         return;
   ptr=loc->next;
   loc->next=ptr->next;
   free(ptr);
}
```

Deleting Entire list

Before the program terminates, the entire list must be deletedso that the memory occupied by the nodes of the list can be used for other purposes. This task can be accomplished by performing the following steps:

- Assign the head pointer to a temporary variable, say ptr.
- Advance the head pointer to the next node.
- Deallocate the memory occupied by the node pointed to by ptr.

The above steps are repeated till the entire list is deleted.

Deleting Entire list

```
Void deletelist(node **head)
{
   node *ptr;
   while(*head!=NULL)
   {
        ptr=*head;
        *head=(*head)->next;
        free(ptr);
   }
}
```

Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Lecture-12

Avinash K. Sharma
Asst. Professor, Department of CSE

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES (KCS301)

TOPIC: DOUBLY LINKED LIST

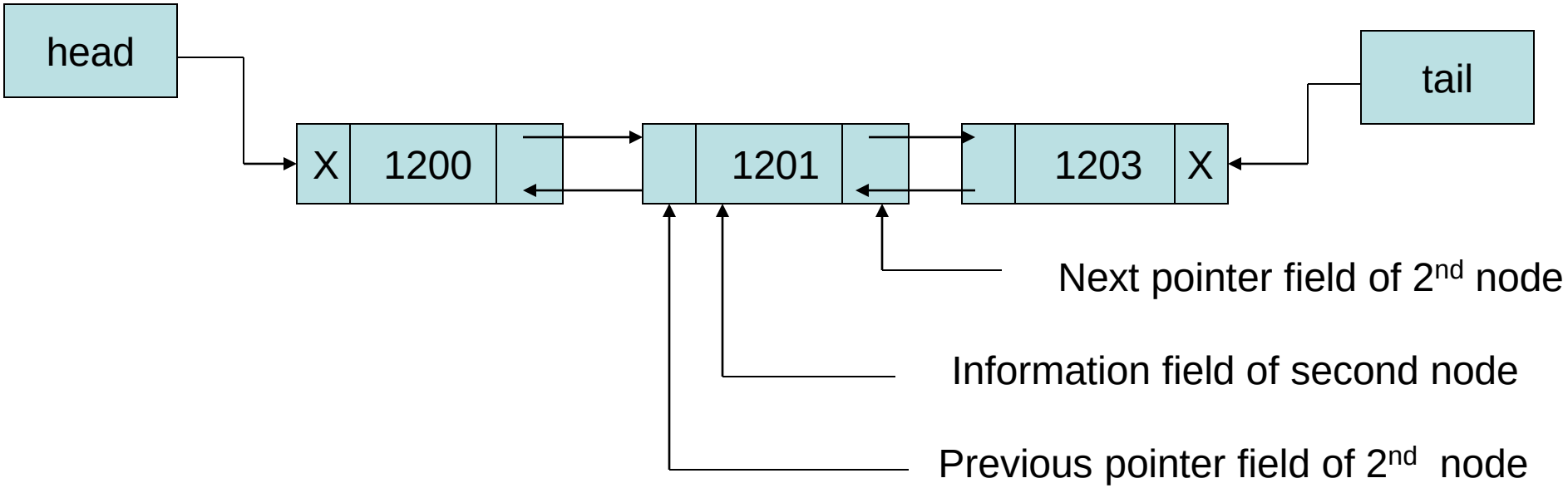
Doubly Linked List

In doubly linked list, also called the two way list, each node is divided into three parts:

- The first part called, previous pointer field, contains the address of preceding element in the list.
- The second part contains the information of the list.
- The third part, called next pointer field, contains the address of the succeeding element in the list.

In addition, two pointer variables, e.g. head and tail, are used that contains the address of first element and the address of last element of the list.

Doubly Linked List



Representation of doubly linked list

- Suppose we want to store list of integer.

```
typedef struct nodetype
        {
                struct nodetype *prev;
                int info;
                struct nodetype *next;
        }node;
node *head,*tail;
```

The above declaration defines a new data type, whose each element is of type nodetype and gives it name node.

Operation on Doubly linked lists

- **Creating an empty list**
- **Traversing a list**
- **Searching an element**
- **Inserting an element**
- **Deleting an element**

Creating an Empty list

- In the previous declaration, the variable head and tail are declared as pointer to a node data type.
- These Variables are not yet given a value.
- The head is used to point to the first element of the list and tail is used to point to the last element of the list.
- Since the list will be empty in the beginning, the variable head and tail are assigned a sentinel value to indicate the list is empty.

```
void createemptylist(node **head, node **tail)
{
        *head=*tail=NULL;
}
```

Traversing a list

Doubly linked list can be traversed in both way and that too very conveniently.

- In order traversal
- Reverse order traversal

In order traversal:

To traverse the doubly linked list, we move along the pointer, and process each element till we reach the last element.

```
void traverseinorder(node *head)
{
        while(head!=NULL)
        {
                printf("%d\n",head->info);
                head=head->next;
        }
}
```

Traversing a list

Reverse order traversal:

The following listing shows the various steps required for traversing a doubly linked list in the backward direction.

```
Void traversereverseorder(node *tail)
        {
                if(tail!=NULL)
                {
                        printf("%d\n",tail->info);
                        tail=tail->prev;
                }
        }
```

Thanks

Inserting an element

To insert an element in the list, the first task is to get a free node, assign the element to be inserted to the info field of the node, and then new node is placed at the appropriate position by adjusting the appropriate pointer. The insertion in the list can take place at the following positions:

- **At the beginning of the list**
- **At the end of the list**
- **After a given element**
- **Before a given element**

Inserting an element

Insert at the beginning of the list:

First test whether the linked list is initially empty, if yes, then the element is inserted as the first and only one element by performing the following steps:

- Assign NULL value to the next pointer and prev pointer field of the new node
- Assign address of new node to head and tail pointer variables.

If the list is not empty, then the element is inserted as the first element of the list by performing the following steps:

- Assign NULL value to the prev pointer field of the new node.
- Assign value of head variable (the address of the first element of the existing list) to the next pointer field of the new node.
- Assign address of the new node to prev pointer field of the node currently pointed by head variable, i. e. first element of the existing list.
- Finally Assign address of the new node to the head variable.

Inserting an element

Insert at the beginning of the list:

```
Void insertatbegining (node **head, node **tail, int item)
    {
        node *ptr;
        ptr=(node*)malloc(sizeof(node));
        ptr->info=item;
        if(*head==NULL)
            ptr->next=ptr->prev=NULL;
            *head=*tail=ptr;
        else
        {
            ptr->prev=NULL;
            ptr->next=*head;
            (*head)->prev=ptr;
            *head=ptr;
        }
}
```

Inserting an element

Inserting at the end of the list

First test whether the linked list is initially empty, if yes, then the element is inserted as the first and only one element by performing the following steps:

- Assign NULL value to the next pointer and prev pointer field of the new node
- Assign address of new node to head and tail pointer variable.

If the list is not empty, then element is inserted as the last element of the list by performing the following steps:

- Assign NULL value to the next pointer field of the new node.
- Assign value of the tail variable (the address of the last element of the existing list) to the prev pointer field of the new node.
- Assign address of the new node to the next pointer field of the node currently pointed by tail variable i.e last element of the existing list.
- Finally assign the address of the new node to tail variable.

Inserting an element

Insert at the end of the list:

```
Void insertatend (node **head, node **tail, int item)
    {
        node *ptr;
        ptr=(node*)malloc(sizeof(node));
        ptr->info=item;
        if(*head==NULL)
                ptr->next=ptr->prev=NULL;
                *head=*tail=ptr;
        else
        {
                ptr->next=NULL;
                ptr->prev=*tail;
                (*tail)->next=ptr;
                *tail=ptr;
        }
}
```

Inserting an element

Inserting after a given element:

```
Void insert afterelement (node **head, node **tail, int item, int after)
    {
        node *ptr, *loc;
        ptr=head;
        loc=search(ptr,after);
        if(loc==NULL)
            return;
        ptr=(node*)malloc(sizeof(node));
        ptr->info=item;
        if(loc->next==NULL)
        {
            ptr->next=NULL;
            loc->next=ptr;
            ptr->prev=*tail;
            *tail=ptr;
        }
        else
        {
            ptr->prev=loc;
            ptr->next=loc->next;
            (loc->next)->prev=ptr;
            loc->next=ptr;
        }
}
```

Inserting an element

Inserting before a given element:

```
Void insertbeforeelement (node **head, node **tail, int item, int before)
    {
        node *ptr, *loc;
        ptr=head;
        loc=search(ptr,before);
        if(loc==NULL)
            return;
        ptr=(node*)malloc(sizeof(node));
        ptr->info=item;
        if(loc->prev==NULL)
        {
            ptr->prev=NULL;
            loc->prev=ptr;
            ptr->next=*head;
            *head=ptr;
        }
        else
        {
            ptr->prev=loc->prev;
            ptr->next=loc;
            (loc->prev)->next=ptr;
            loc->prev=ptr;
        }
}
```

Deleting an element

- To delete an element from the list, first the pointer are set properly and then the memory occupied by the node to be deleted is deallocated (freed).
- The deletion in the list can take place at the following positions.

1. At the beginning of the list
2. At the end of the list
3. After a given element
4. Before a given element

Deleting an element

Deleting from the beginning of the list:

An element from the beginning of the lists can be deleted by performing following steps:

- Assign the value of head (address of the first element of the list) to a temporary variable (say ptr)
- Further there are two cases:

1. If there is only one element in the existing list, both head and tail variable are set to NULL.
2. If there are more than one element in the list then following steps are given below:
   - Assign NULL value to the prev pointer field of the second node.
   - Assign address of the second node to head.
   3. Deallocate the memory occupied by the node pointed to by ptr.

Deleting an element

Deleting from the beginning of the list:

```
Void deletefrombegining( node **head, node **tail)
{
    node *ptr;
    if(*head==NULL)
        return;
    ptr=*head;
    if(*head==*tail)      /*one element only*/
        *head=*tail=NULL;
    else
    {
        (ptr->next)->prev=NULL;
        *head=ptr->next;
    }
    free(ptr);
}
```

Deleting an element

Deleting from the end of the list:

An element from the end of the list can be deleted by performing following steps:

- Assign the value of tail (address of the last element of the list) to a temporary variable (say ptr)
- Further there are two cases:

1. If there is only one element in the existing list, both head and tail variable are set to NULL.
2. If there are more than one element in the list then following steps are given below:
   - Assign NULL value to the next pointer field of the second last node.
   - Assign address of the second last node to tail.
   3. Deallocate the memory occupied by the node pointed to by ptr.

Deleting an element

Deleting from the end of the list:

```
Void deletefromend( node **head, node **tail)
{
    node *ptr;
    if(*head==NULL)
        return;
    ptr=*tail;
    if(*head==*tail)      /*one element only*/
        *head=*tail=NULL;
    else
    {
        (ptr->prev)->next=NULL;
        *tail=ptr->prev;
    }
    free(ptr);
}
```

Deleting an element

Deleting after a given element:

```
Void ideleteafterelement (node **head, node **tail, int item, int after)
    {
        node *ptr, *loc;
        ptr=head;
        loc=search(ptr,after);
        if(loc==NULL)
            return;
        else if((loc->next)->next==NULL)
            {
            ptr=loc->next;
            loc->next=NULL;
            *tail=loc;
            free(ptr);
            }
        else
            {
            ptr=loc->next;
            loc->next=ptr->next;
            (ptr->next)->prev=loc;
            free(ptr);
            }
}
```

Deleting an element

Deleting before a given element:

```
Void ideleteafterelement (node **head, node **tail, int item, int before)
    {
        node *ptr, *loc;
        ptr=head;
        loc=search(ptr,before);
        if(loc==NULL)
            return;
        else if((loc->prev)->prev==NULL)
            {
            ptr=loc->prev;
            loc->prev=NULL;
            *head=loc;
            free(ptr);
            }
        else
            {
            ptr=loc->prev;
            loc->prev=ptr->prev;
            (ptr->prev)->next=loc;
            free(ptr);
            }
}
```

Deleting entire list

The doubly linked list can be deleted either by heading from the beginning or from the end. The list can be deleted from the beginning by performing the following steps:

- Assign the head pointer to a temporary variable, say ptr.
- Advance the head pointer to the next node.
- Deallocate the memory occupied by the node pointed to by ptr.

The above steps are repeated till the entire list is deleted. Finally the tail pointer is set to NULL value.

Deleting entire list

```
Void deletelist(node **head, node **tail)
{
    node *ptr;
    while(*head!=NULL)
    {
        ptr=*head;
        *head=(*head)->next;
        free(ptr);
    }
    *tail=NULL;
}
```

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Lecture-13

Avinash K. Sharma
Asst. Professor, Department of CSE

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES (KCS301)

TOPIC: CIRCULAR LINKED LIST

Circular Linked List

A circular list is a linear linked list, except that the last element points to the first element. For non empty circular linked list there are no NULL pointer.

The memory declarations for representing circular linked lists are the same as for linear linked lists

Properties:

- **Can reach entire list from any node**
- **Need special test for end of list**
- **Used as buffer**

Circular Linked List

- A Circular Linked List is a special type of Linked List
- It supports traversing from the end of the list to the beginning by making the last node point back to the head of the list.
- Circular linked lists are usually sorted
- Circular linked lists are useful for playing video and sound files in “looping” mode.
- They are also a stepping stone to implementing graphs, an important topic in computer graphics.

Representation of Circular linked list

Suppose we want to store the list of integer numbers, then the circular linked list can be represented in memory with the following declarations.

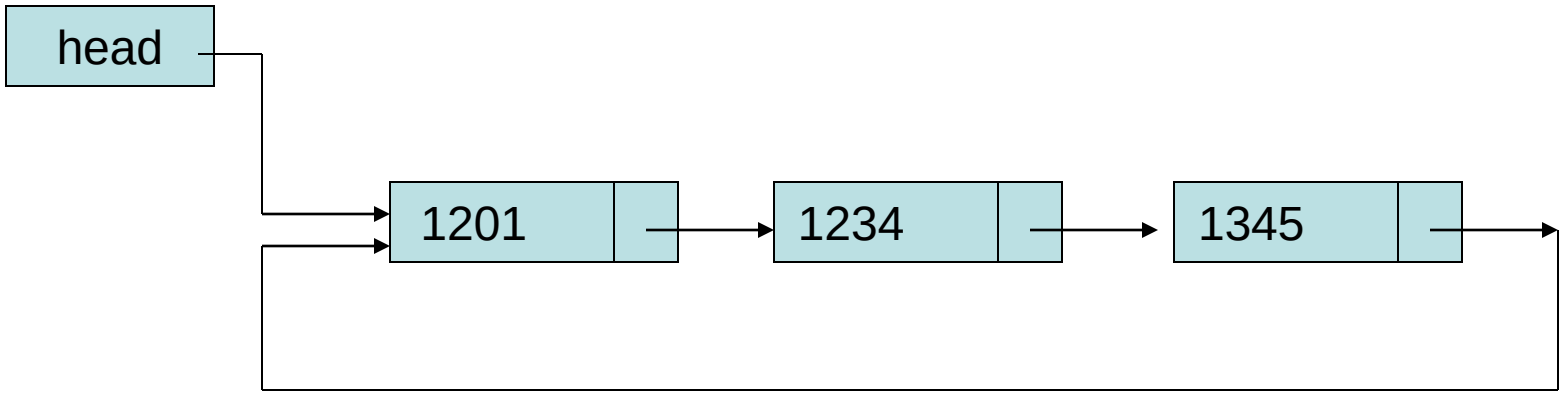
```
typedef struct nodetype
{
        int info;
        struct nodetype *next;
}node;
node *head;
```

The above declaration define a new data type, whose each element is of type nodetype and gives it a name node.

Circular Linked List

All operations performed on linear linked list can be easily extended for circular linked lists with following exceptions:

1. While inserting new node at the end of the lists, its next pointer field is made to point to the first node.
2. While testing for the end of the lists, we compare the next pointer field with the address of the first node.



Circular Linked with 3 Nodes

Thanks

Operation on Linear linked lists

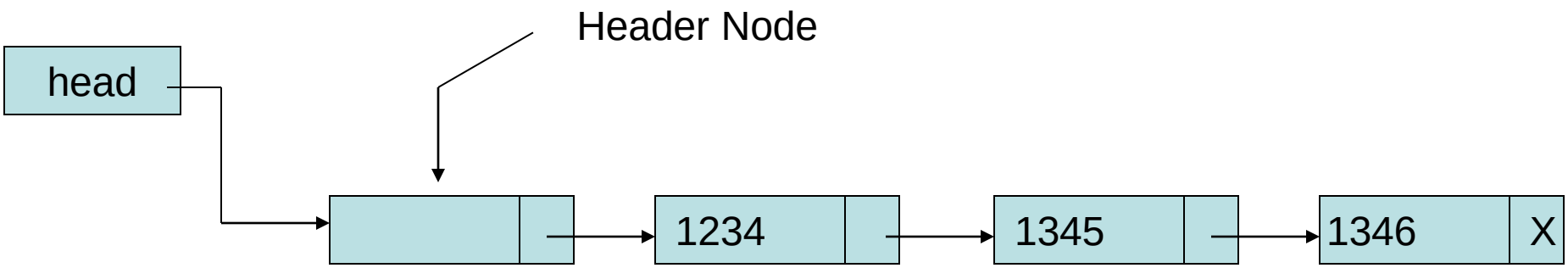
- **Creating an empty list**
- **Traversing a list**
- **Searching an element**
- **Inserting an element**
- **Deleting an element**

Header Linked List

- **A header list is a linked list, which always contains a special node, called header node, at the beginning of the linked list.**

- **This header node usually contains vital information about the linked list such as the number of nodes in the list, whether the list is sorted or not.**

Header Linked List

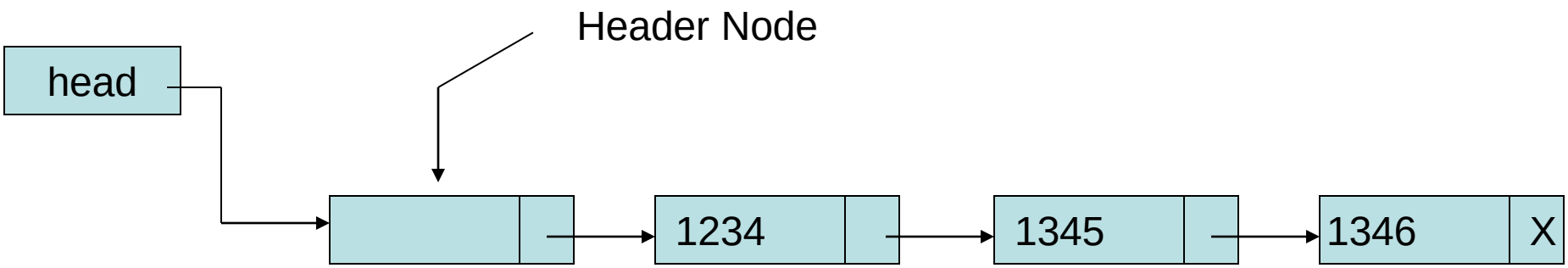


Header Linked List

Types of header linked list:

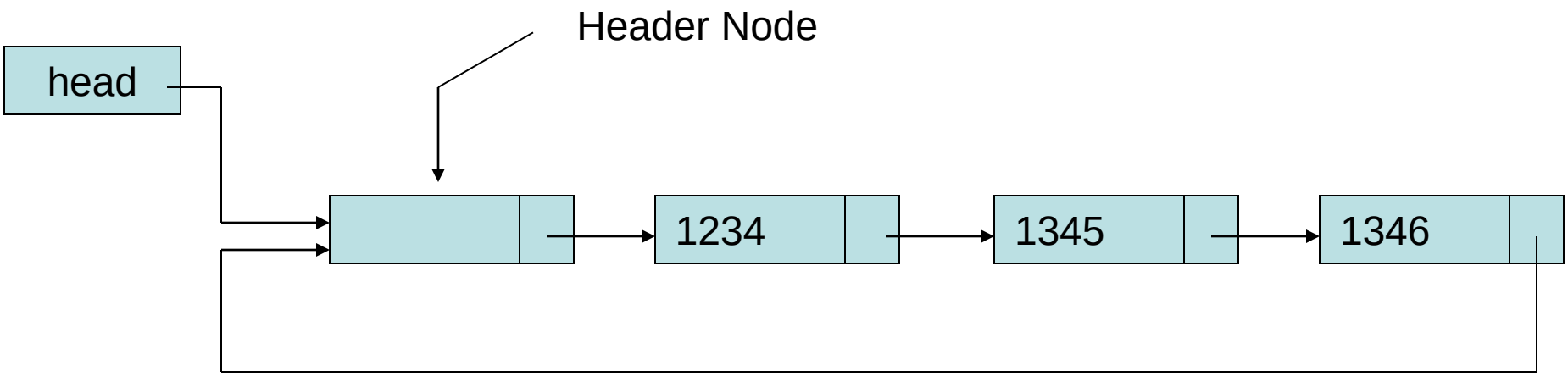
- **Header linear linked list**
- **Circular header list**
- **Two way header list**
- **Two way circular header list**

Header Linked List



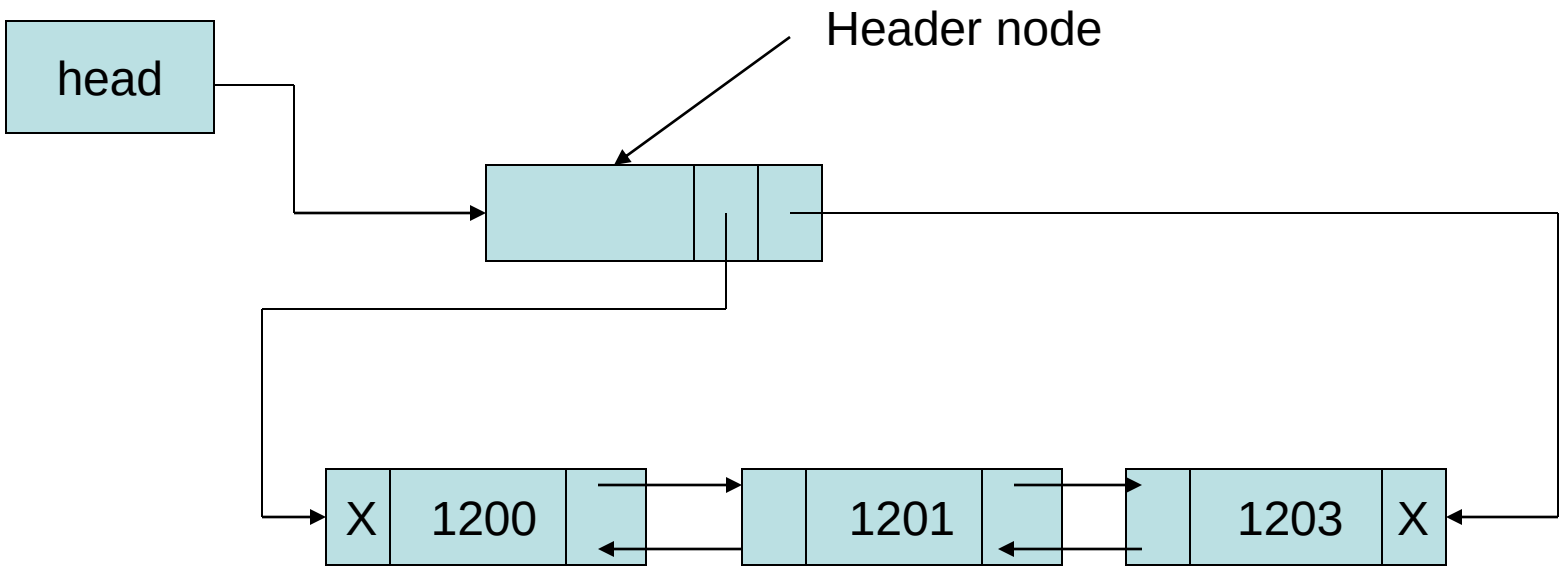
Header linear linked list

Header Linked List



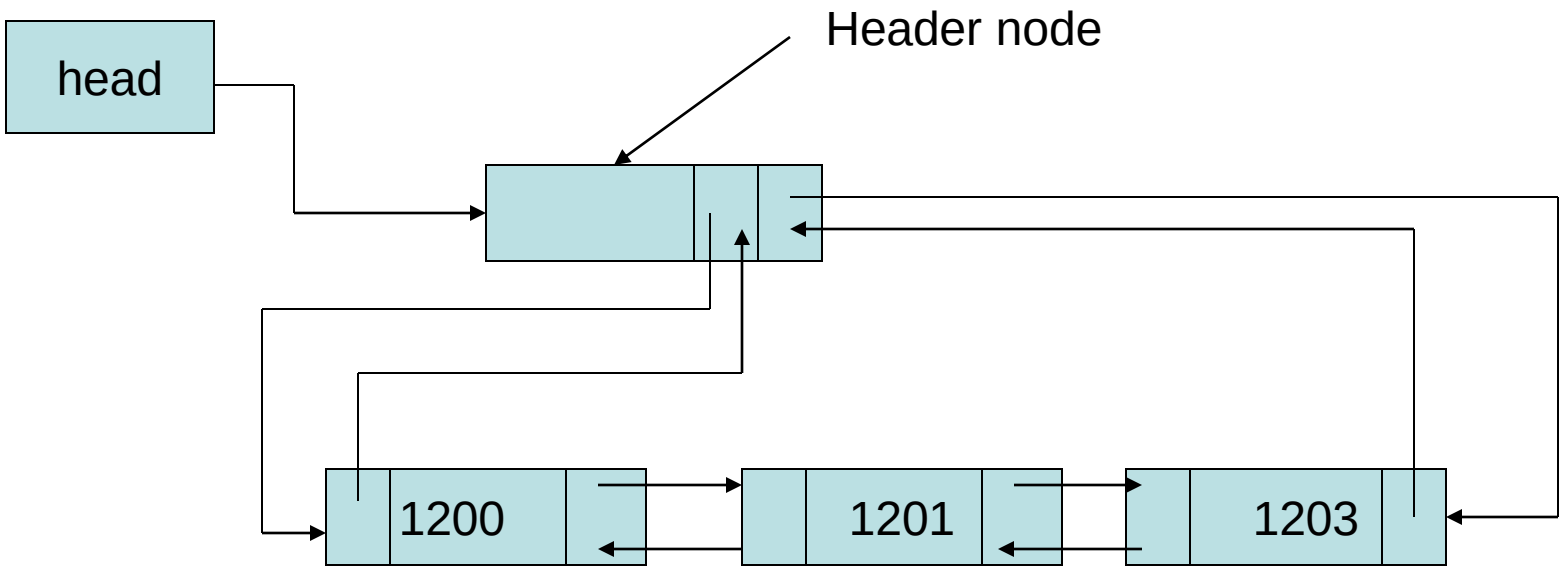
Circular header list

Header Linked List



Two way header list

Header Linked List



Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Lecture-16

Avinash K. Sharma
Asst. Professor, Department of CSE

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES (KCS301)

TOPIC: APPLICATIONS OF LINKED LIST

Applications of linked lists

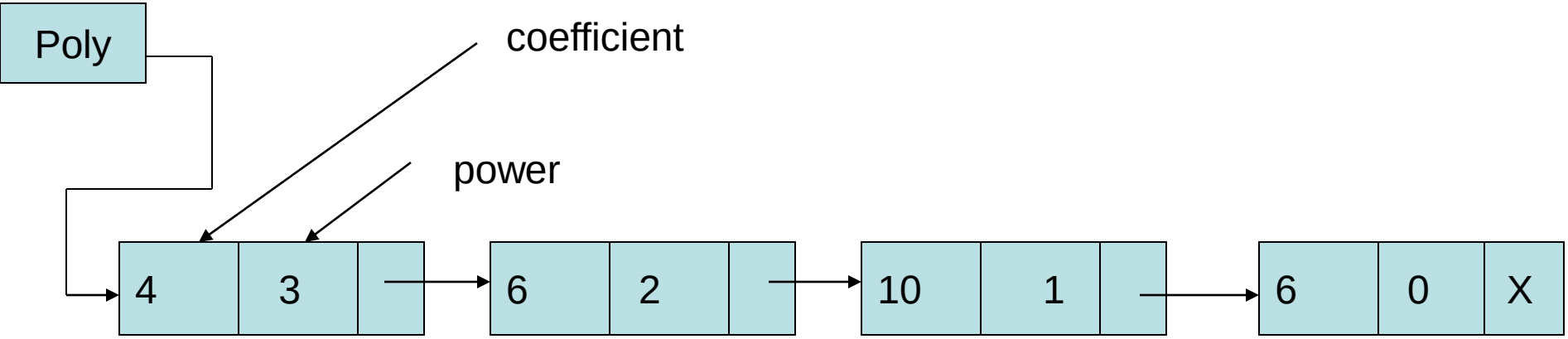
- To implement the other data structures such as stacks, queues, trees and graphs.
- To maintain a directory of names.
- To perform arithmetic operation on long integers.
- To manipulate polynomial.
- To represent sparse matrices.

Polynomial Manipulation

A polynomial of type

$$4x^3+6x^2+10x+6$$

can be represented using following linked list



In the above list, each node has the following structure

Coefficient of the term	Power of x	Link to the next node

Polynomial Manipulation

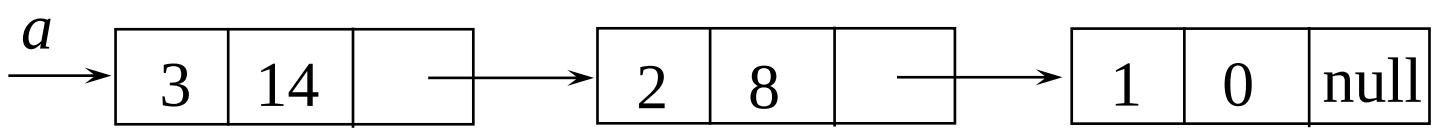
The required memory declarations for the representation of a polynomial with integer coefficients are

```
typedef struct nodetype
    {
        int coeff;
        int power;
        struct nodetype *next;
    }node;
    node *poly;
```

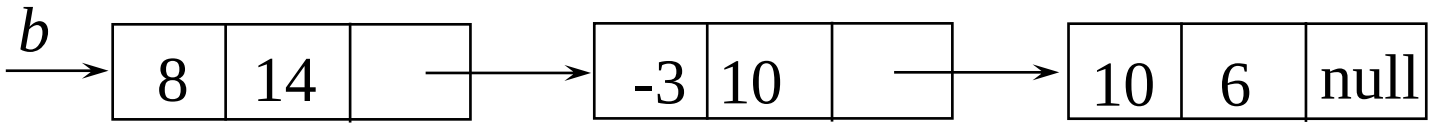
Polynomial Representation: Example

- **Examples**

$$a = 3x^{14} + 2x^{8} + 1$$



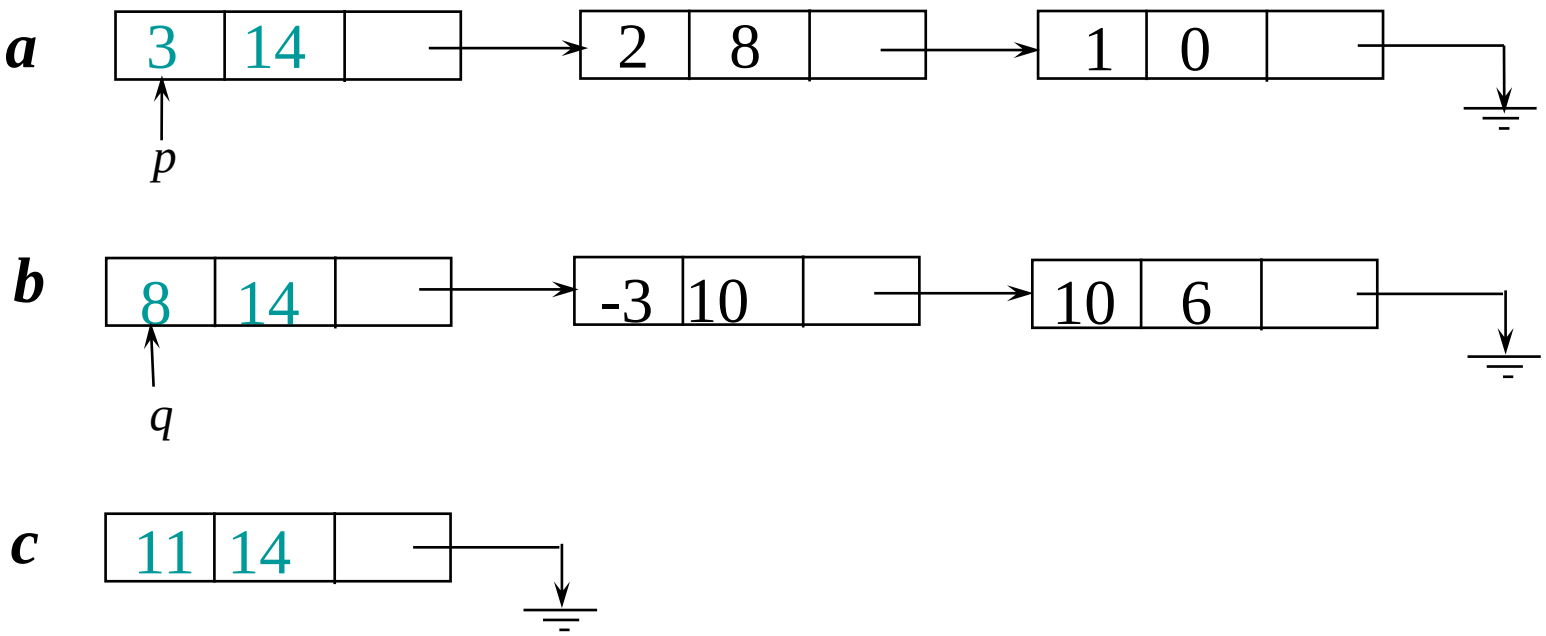
$$b = 8x^{14} - 3x^{10} + 10x^{6}$$



4.6.2 Adding Polynomials: $c = a + b$

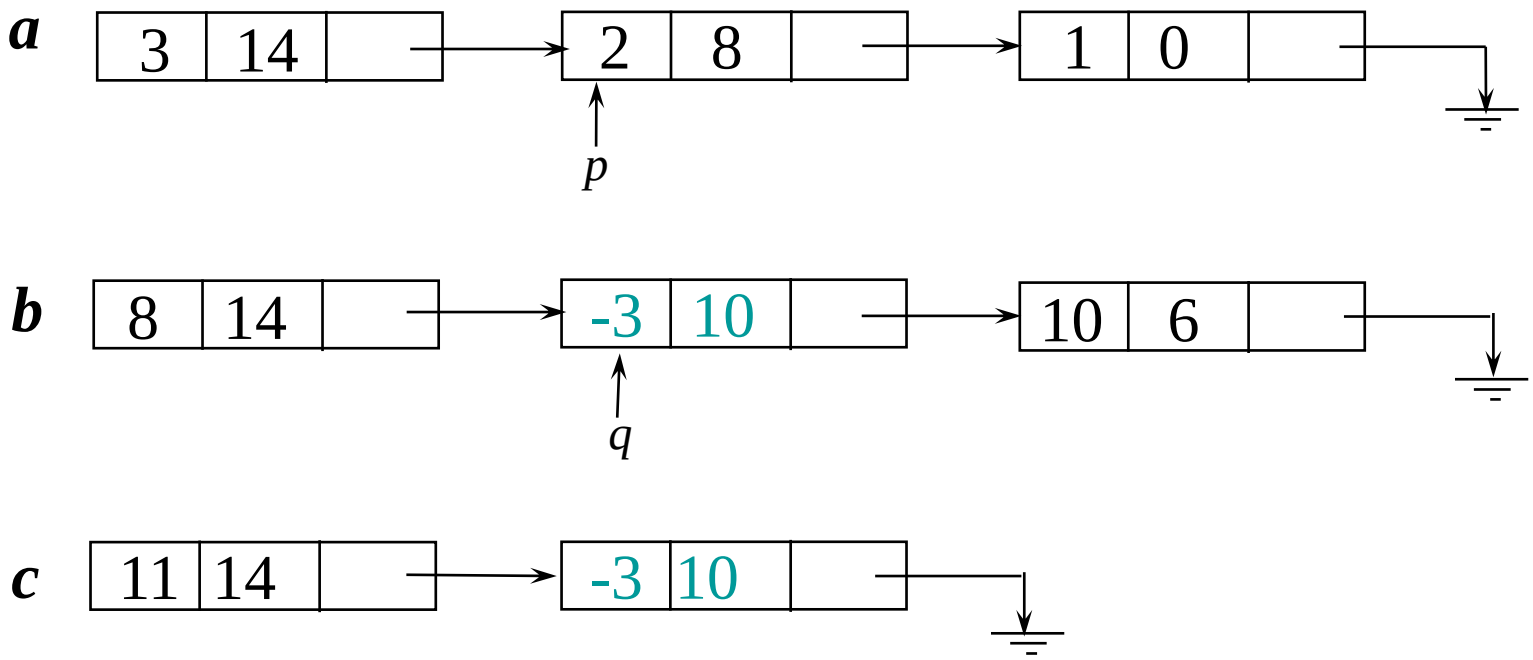
- **Adding Polynomials: Figure 4:19 $c = a + b$**

Case 1: *p->exp* = *q->exp*



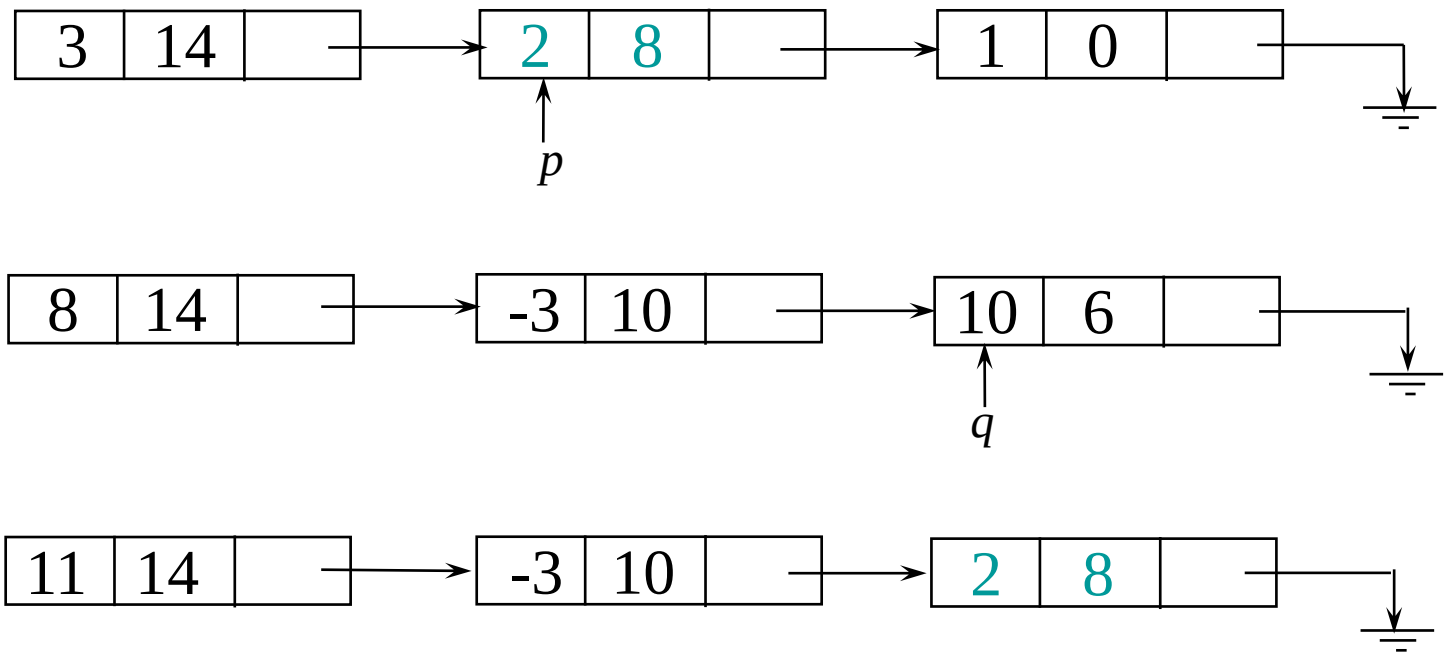
Adding Polynomials : $c = a + b$ (cont.)

Case 2: *p->exp* < *q->exp*



Adding Polynomials: $c = a + b$ (cont.)

Case 3: *p->exp* > *q->exp*



Representing sparse matrices

$$\begin{pmatrix} 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 5 \\ 0 & 4 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 7 & 0 \end{pmatrix}$$

A 5X6 sparse matrices

- inadequate of sequential schemes

(1) # of nonzero terms will vary after some matrix computation
(2) matrix just represents intermediate results

- new scheme
- Each column (row): a circular linked list with a head node

Sparse Matrix

The description of this representation is as

- It contains one header node that has four fields….

  --#of rows

  --#of cols

  --#of non zero elements

  --head i.e. pointer to 1st row containing at least one non zero element.

- A linked list of rows containing at least one non zero term, in the ascending order of their row values. each node of this list has three fields

  --row (row number for corresponding row list)

  --next (pointer to next node in the row list)

  --first (a pointer to first column in a row having non zero item)

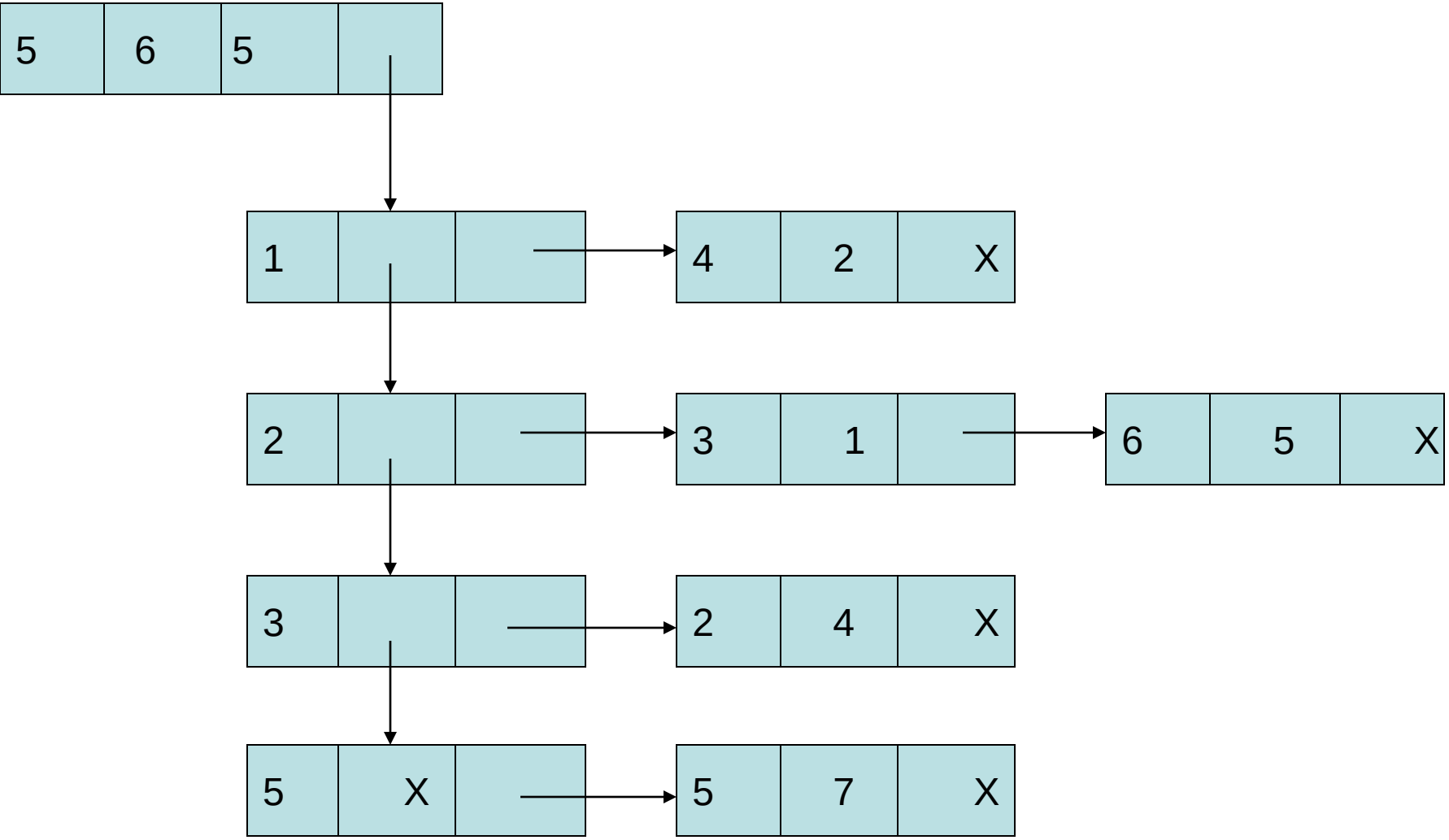
- A linked list of columns containing nonzero terms, in the ascending order of their column values. Each node of this list has three fields

  --col (column number for corresponding row list)

  --term (a non zero value in column col)

  --link (a pointer to next column having non zero element)

Sparse Matrix



A linked representation of sparse matrix

Generalized Linked List

A Generalized Linked List L, is defined as a finite sequence of n>=0 elements, $l_1, l_2, l_3, l_4, \ldots, l_n$, such that l_i are either atom or the list of atoms. Thus

$$\mathbf{L = (l_1, l_2, l_3, l_4, \ldots, l_n)}$$

where n is total number of nodes in the list.

To represent a list of items there are certain assumptions about the node structure.

Flag = 1 implies that *down pointer* exists
Flag = 0 implies that *next pointer* exists
Data means the atom
Down pointer is the address of node which is down of the current node
Next pointer is the address of node which is attached as the next node

Why Generalized Linked List?

Generalized linked lists are used because although the efficiency of polynomial operations using linked list is good but still, the disadvantage is that the linked list is unable to use *multiple variable polynomial equation* efficiently. It helps us to represent multi-variable polynomial along with the list of elements.

Assignment-Write Programs in C

- Insertion and deletion in singly linked list for all the cases with traversal.
- Insertion and deletion in doubly linked list for all the cases with traversal.
- Ploynomial representation and addition of two polynomial equations.
- Take any 5*6 sparse matrix and represent with linked list.

Thanks